

Oxide Surface Reactivity Workflow — Design Document (v0.2)

Author: Sean · **Status:** Prototype scope, pre-coding **Scope note:** v0.2 narrows v0.1 to the benchmarking prototype: per-stage and full-pipeline relaxation divergence between MLIPs and a reference. Screening, DFT job preparation, magnetic enumeration, and facet screening are deferred (§8). The long-term architecture from v0.1 is retained where it costs nothing.

1. Problem statement

Computing surface reaction energetics (vacancy formation, adsorption) on metal oxides requires a chain of structure preparation: bulk → surface → defect → adsorbate → energies. Existing tools cover fragments (WhereWulff, AdsorbML/OCData, pymatgen); nothing covers the full chain for reducible oxides with defect-mediated (MvK) chemistry while treating MLIP-vs-DFT switching and benchmarking as first-class operations.

This prototype answers one question: **when an MLIP performs each relaxation stage itself, where and how does it diverge from a reference — per stage, and accumulated over the full pipeline?** DFT is fixed ground truth in this phase; improving or extending DFT is downstream of the error map this produces.

Domain: metal oxides broadly. Test: no stage's code mentions "perovskite."

2. Use cases (prototype)

1. **Per-stage divergence (primary).** For each stage, the candidate model builds its starting structure from the *reference's* relaxed previous stage and performs the stage's relaxation itself. Divergence is attributable to that stage alone. Answers: *which relaxation breaks, and how.*
2. **Full-pipeline divergence (secondary).** The candidate model runs end-to-end, each stage feeding its own output forward. Answers: *how bad is the realistic accumulated error.* The per-stage vs full-pipeline contrast is itself a result (intrinsic vs inherited error).
3. **Reference verification.** Ingest a trusted reference structure mid-pipeline and explore around it (alternative vacancy sites, alternative placements) to check whether the reference found the right minimum.

By-product: every enumerate-relax funnel records all candidate energies per model, so ranking-fidelity evidence accumulates from normal use.

3. Starting-structure rule (the core protocol definition)

"Seeded" refers to which previous-stage output a stage is built from — never to starting at the current stage's answer.

Every stage's starting structure = *previous stage's relaxed output* + *fresh modification*, which is unrelaxed by construction for the current stage:

- slab = cut from relaxed bulk → surface unrelaxed
- vacancy = O removed from relaxed slab → defect unrelaxed
- adsorbate = fragment placed at heuristic height on relaxed substrate → unrelaxed

Both reference and candidate models perform the full relaxation work of every stage. The only warm input in the chain is the bulk starting from the database structure — unavoidable, shared by both models, fully re-relaxed by each.

Mode	Stage input comes from	Measures
Seeded per-stage	reference's relaxed previous stage	intrinsic per-stage error
Full-pipeline	candidate's own relaxed previous stage	realistic accumulated error
(Basin diagnostic, optional)	reference's relaxed <i>same</i> stage	basin stability only — never the benchmark (flattery trap)

4. Pipeline stages (prototype chain)

Stages follow **enumerate** → **relax** → **select/carry**. Structures are first-class inputs at every stage boundary.

1. Bulk MP-ID input (relaxed-elsewhere warm start; RMSD from input logged)
2. Slab facet + termination pinned by config; cut, relax
3. Vacancy symmetry-distinct O sites on relaxed slab; remove representative; relax
4. Adsorbate symmetry-enumerate sites (pymatgen AdsorbateSiteFinder);

relax ALL candidates with the reference model; minimum defines the

canonical adsorbate stage; every candidate's energy + structure recorded

5. Assembly reaction recipes (YAML, structures by role); atom-balance hard gate;

gas references per backend; referencing scheme recorded per ΔE

Adsorbate placement decision: AdsorbML's philosophy (test many, keep the minimum) without AdsorbML's dependencies. Placement matters only at reference generation; the candidate model is seeded from stage inputs, not placements. Saved candidate energies from both models are the ranking-fidelity by-product.

Vacancy/adsorbate share one interface: `decorate(substrate, modification) → symmetry-reduced candidates`. Site identity is stored as symmetry class + fractional coordinate, never file line numbers.

Match mode (for real DFT references): map pristine↔defected reference structures under PBC to extract the *site identity* only — never the relaxed coordinates — then rebuild from the pipeline's own structures. Verify substrate equivalence first; refuse loudly on mismatch.

5. Outputs

Primary: per-stage divergence table (seeded mode). One row per (composition, stage, model). **Secondary: full-pipeline table**, same schema. Canonical divergence record:

{composition, polymorph, facet, termination, stage, model, geometry_source, protocol,

start_fmax_at_ref_geom, # force felt at the stage input before relaxing

mean_displacement, # uniform-drift baseline

rmsd, # StructureMatcher-aligned, large-move weighted

max_displacement, max_disp_atom, # localization: worst atom + identity

energy_error, # candidate energy at ITS OWN minimum vs reference energy

[active_site_dBO, symmetry_match]} # chemistry-aware layer, metadata-gated

The displacement triple comes from one per-atom displacement vector, computed after StructureMatcher alignment under PBC. $\text{rmsd} \approx \text{mean} \Rightarrow$ uniform drift; $\text{rmsd} \gg \text{mean} \Rightarrow$ localized failure; `max_disp_atom` names the culprit.

Derived — trend fidelity: per stage, Spearman rank correlation between reference and candidate relaxed energies across compositions. "Systematically off but rank-correct" and "rank-wrong" are opposite verdicts; MAE alone cannot distinguish them.

Tables are long-format so every analysis (per-stage attribution, per-composition error, ranking) is a pivot, not a re-run. All expensive results are written to disk immediately.

6. Backends

All backends sit behind one interface: `relax(structure, backend) → (relaxed structure, energy, trajectory metadata)`.

Hard constraint from day one: subprocess isolation. Models live in mutually incompatible environments (e3nn pin conflict between MACE and fairchem). The orchestrator is env-agnostic: it launches a worker script in the model's environment and reads results from disk (the perovskite-mlip-bench pattern). This constraint *is* the backend abstraction in physical form. Backends carry a capability declaration (`can_relax`, `is_async`, `training_labels`); DFT enters later as an async backend (export job dirs / ingest OUTCARs) behind the same interface.

7. Prototype plan

Material: rutile TiO₂ (mp-2657) — nonmagnetic d⁰, small cells, best-characterized oxide surface, runs on CPU. Validates the *machinery*, not the science; perovskites remain the experiment.

Pseudo-reference plumbing test: MACE-OMAT24 acts as the "reference" (generates the full relaxed chain, stage by stage); UMA runs both modes against it. If the pipeline produces sensible divergence tables comparing two MLIPs, swapping in real VASP data is a data change, not a code change.

Build order (one step at a time, each run before the next):

1. Repo skeleton; this document at root
2. Records: structure record + divergence record as dataclasses, serialized to disk (JSON + POSCAR)
3. `relax()` subprocess-wrapped; green light = rutile bulk relaxes with MACE locally
4. `diverge(struct_A, struct_B) → DivergenceRecord`; unit-tested on a structure vs a rattled copy of itself
5. Stage chain: pseudo-reference generation, then seeded + full-pipeline candidate runs (~glue around tested parts)

8. Deferred (interfaces already accommodate them)

MLIP-first screening · VASP job export/ingest as production feature · magnetic enumeration service (callable-service seam retained) · facet/Wulff screening (facets are pinned config) · CIF/prototype/OPTIMADE bulk sources · dopant substitution · polymorph selection policies beyond pinned · coverage & co-adsorption · AdsorbML-proper integration · spin-constrained backends · fine-tuning arm (Block 2).

9. Open questions

1. Reference dataset completeness: per-stage energies AND relaxed geometries under one consistent protocol (the load-bearing requirement — a dataset with only final ΔE s cannot be stage-decomposed).
2. Which bulk cells and oxygen referencing convention the real DFT reference uses (match-mode behavior depends on both).

Appendix: briefing paragraph for coding sessions

This repo implements the design in [oxide-workflow-design.md](#) (v0.2). Build it one numbered step at a time per §7, running each step before writing the next. Hard constraints: (1) models run via subprocess isolation in their own conda envs (mace-clean for MACE, mlip-mace for fairchem/UMA — they cannot share an env due to an e3nn version conflict); the orchestrator only launches workers and reads results from disk. (2) Every stage's starting structure is the previous stage's relaxed output plus a fresh modification — unrelaxed by construction for the current stage; never start a candidate model at the reference's relaxed geometry of the same stage. (3) All results are dataclass records serialized to disk immediately (JSON + POSCAR); the divergence record schema in §5 is canonical. (4) No stage's code may mention "perovskite" — the prototype material is rutile TiO_2 (mp-2657), and the pipeline must stay material-agnostic. Do not build ahead of the numbered steps; do not add magnetic handling, facet screening, or DFT execution — they are explicitly deferred.